

Section 11.3.3 - Sequence models

December 20, 2025

1 The sequence model approach

- Represent input samples as sequence of integer indices
- Map each integer to a vector to form vector sequences

- 1) Bidirectional LSTMs (2016-2020)
- 2) The Transformers (nowadays)

```
[1]: import tensorflow as tf
from tensorflow import keras

batch_size = 16

train_ds = keras.utils.text_dataset_from_directory(
    "aclImdb/train", batch_size=batch_size
)
val_ds = keras.utils.text_dataset_from_directory(
    "aclImdb/val", batch_size=batch_size
)
test_ds = keras.utils.text_dataset_from_directory(
    "aclImdb/test", batch_size=batch_size
)

text_only_train_ds = train_ds.map(lambda x, y: x)
```

Found 20000 files belonging to 2 classes.

Found 5000 files belonging to 2 classes.

Found 25000 files belonging to 2 classes.

1.1 Bidirectional LSTM with one-hot encoded vector sequence

- One-hot encoding: tokens are independent!
- Sample size (max_length, max_tokens) -> slow
- Test accuracy 87% -> inefficient

```
[2]: from tensorflow.keras import layers

max_length = 600
max_tokens = 20000
```

```

text_vectorization = layers.TextVectorization(
    max_tokens=max_tokens,
    output_mode="int",
    output_sequence_length=max_length,
)
text_vectorization.adapt(text_only_train_ds)

int_train_ds = train_ds.map(
    lambda x, y: (text_vectorization(x), y),
    num_parallel_calls=4)
int_val_ds = val_ds.map(
    lambda x, y: (text_vectorization(x), y),
    num_parallel_calls=4)
int_test_ds = test_ds.map(
    lambda x, y: (text_vectorization(x), y),
    num_parallel_calls=4)

```

```

[3]: inputs = keras.Input(shape=(None,), dtype="int64")
embedded = tf.one_hot(inputs, depth=max_tokens, dtype=tf.float32)

x = layers.Bidirectional(layers.LSTM(32))(embedded)
x = layers.Dropout(0.5)(x)
outputs = layers.Dense(1, activation="sigmoid")(x)

model = keras.Model(inputs, outputs)
model.compile(optimizer="rmsprop",
              loss="binary_crossentropy",
              metrics=["accuracy"])
model.summary()

```

Model: "model"

Layer (type)	Output Shape	Param #
input_1 (InputLayer)	[(None, None)]	0
tf.one_hot (TFOpLambda)	(None, None, 20000)	0
bidirectional (Bidirectional)	(None, 64)	5128448
dropout (Dropout)	(None, 64)	0
dense (Dense)	(None, 1)	65

Total params: 5128513 (19.56 MB)
 Trainable params: 5128513 (19.56 MB)

Non-trainable params: 0 (0.00 Byte)

Training the basic sequence model

```
[4]: # Hide the warning messages
import tensorflow as tf
import logging

tf.get_logger().setLevel(logging.ERROR)

import warnings
warnings.filterwarnings(
    "ignore",
    message="You are saving your model as an HDF5 file.*"
)

callbacks = [
    keras.callbacks.ModelCheckpoint("one_hot_bidir_lstm.h5",
                                    save_best_only=True)
]
history = model.fit(int_train_ds, validation_data=int_val_ds, epochs=10,
                    ↪callbacks=callbacks)
model = keras.models.load_model("one_hot_bidir_lstm.h5")
print(f"Test acc: {model.evaluate(int_test_ds)[1]:.3f}")
```

Epoch 1/10

1250/1250 [=====] - 128s 100ms/step - loss: 0.5350 -
accuracy: 0.7309 - val_loss: 0.3416 - val_accuracy: 0.8604

Epoch 2/10

1250/1250 [=====] - 126s 101ms/step - loss: 0.3464 -
accuracy: 0.8687 - val_loss: 0.2969 - val_accuracy: 0.8860

Epoch 3/10

1250/1250 [=====] - 126s 101ms/step - loss: 0.2890 -
accuracy: 0.8961 - val_loss: 0.5705 - val_accuracy: 0.7758

Epoch 4/10

1250/1250 [=====] - 127s 101ms/step - loss: 0.2539 -
accuracy: 0.9133 - val_loss: 0.3051 - val_accuracy: 0.8862

Epoch 5/10

1250/1250 [=====] - 127s 102ms/step - loss: 0.2272 -
accuracy: 0.9228 - val_loss: 0.2912 - val_accuracy: 0.8902

Epoch 6/10

1250/1250 [=====] - 127s 102ms/step - loss: 0.2073 -
accuracy: 0.9312 - val_loss: 0.3227 - val_accuracy: 0.8894

Epoch 7/10

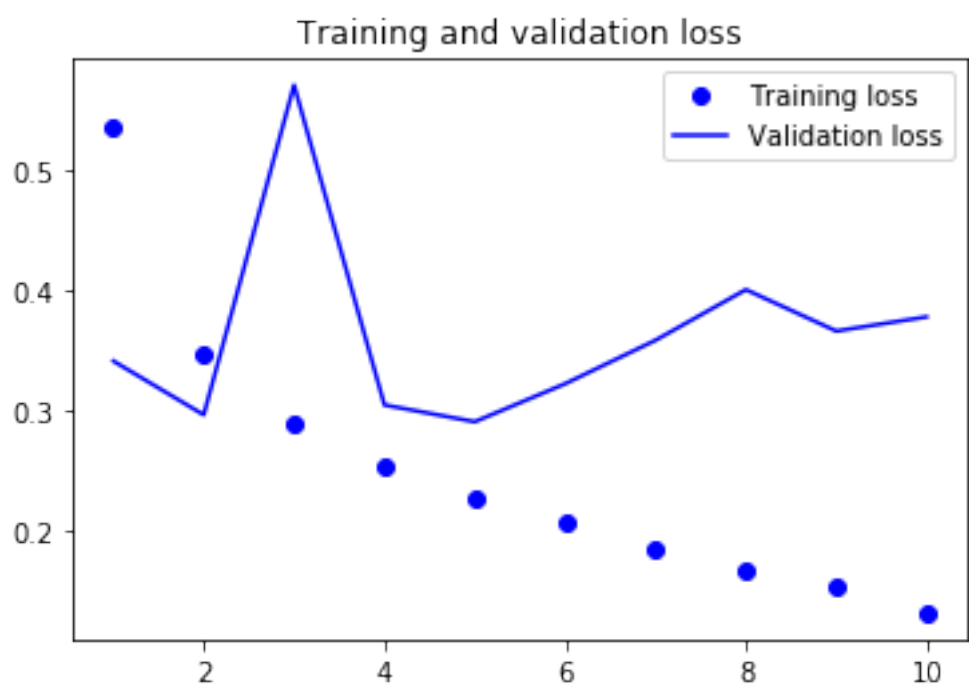
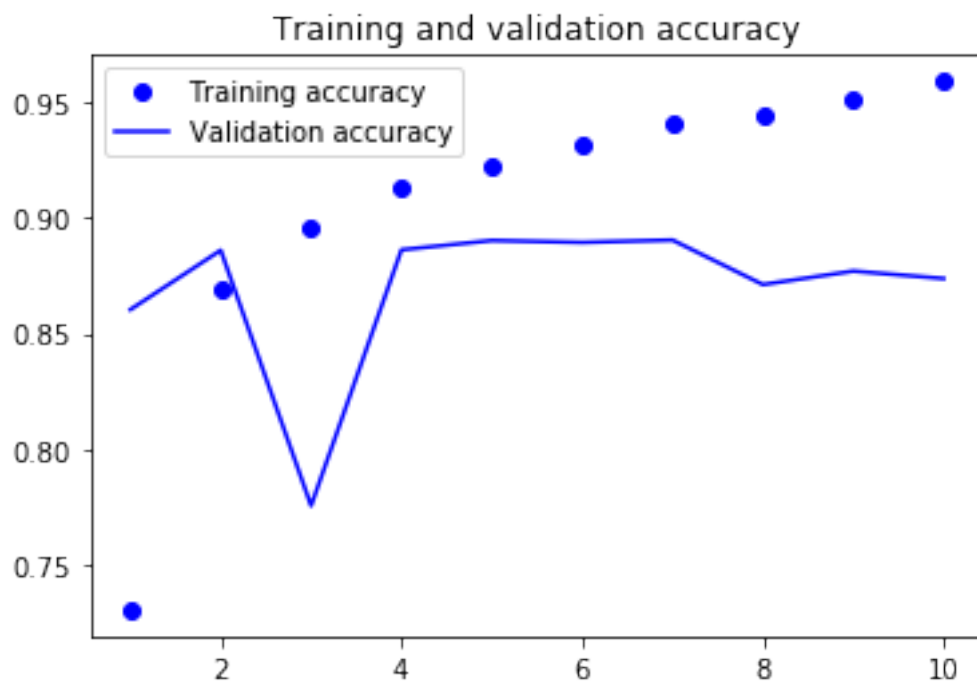
1250/1250 [=====] - 127s 102ms/step - loss: 0.1866 -
accuracy: 0.9410 - val_loss: 0.3588 - val_accuracy: 0.8904

Epoch 8/10

```
1250/1250 [=====] - 127s 102ms/step - loss: 0.1683 -  
accuracy: 0.9447 - val_loss: 0.4008 - val_accuracy: 0.8712  
Epoch 9/10  
1250/1250 [=====] - 127s 102ms/step - loss: 0.1552 -  
accuracy: 0.9516 - val_loss: 0.3664 - val_accuracy: 0.8770  
Epoch 10/10  
1250/1250 [=====] - 127s 102ms/step - loss: 0.1325 -  
accuracy: 0.9589 - val_loss: 0.3782 - val_accuracy: 0.8738  
1563/1563 [=====] - 82s 52ms/step - loss: 0.3173 -  
accuracy: 0.8721  
Test acc: 0.872
```

Plotting the results

```
[5]: import matplotlib.pyplot as plt  
  
acc = history.history["accuracy"]  
val_acc = history.history["val_accuracy"]  
loss = history.history["loss"]  
val_loss = history.history["val_loss"]  
epochs = range(1, len(acc) + 1)  
plt.plot(epochs, acc, "bo", label="Training accuracy")  
plt.plot(epochs, val_acc, "b", label="Validation accuracy")  
plt.title("Training and validation accuracy")  
plt.legend()  
plt.figure()  
plt.plot(epochs, loss, "bo", label="Training loss")  
plt.plot(epochs, val_loss, "b", label="Validation loss")  
plt.title("Training and validation loss")  
plt.legend()  
plt.show()
```



1.2 Using an Embedding layer

- Geometric relationship between vectors should reflect semantic relationship between the words
- Map human language into a structured geometric space
- Dense, low-dimensional, floating-point vectors
- As a layer - vectors are learnt from data
- Specific directions in the embedding space are meaningful/interpretable
- Good word embedding space depends on task
- input (batch_size, sequence_length)
- output (batch_size, sequence_length, embedding_dimensionality)

```
[6]: embedding_layer = layers.Embedding(input_dim=max_tokens, output_dim=256)
```

```
[7]: inputs = keras.Input(shape=(None,), dtype="int64")
embedded = layers.Embedding(input_dim=max_tokens, output_dim=256)(inputs)
x = layers.Bidirectional(layers.LSTM(32))(embedded)
x = layers.Dropout(0.5)(x)
outputs = layers.Dense(1, activation="sigmoid")(x)
model = keras.Model(inputs, outputs)
model.compile(optimizer="rmsprop",
              loss="binary_crossentropy",
              metrics=["accuracy"])
model.summary()

callbacks = [
    keras.callbacks.ModelCheckpoint("embeddings_bidir_gru.h5",
                                   save_best_only=True)
]
history = model.fit(int_train_ds, validation_data=int_val_ds, epochs=10,
                  callbacks=callbacks)
model = keras.models.load_model("embeddings_bidir_gru.h5")
print(f"Test acc: {model.evaluate(int_test_ds)[1]:.3f}")
```

Model: "model_1"

Layer (type)	Output Shape	Param #
input_2 (InputLayer)	[(None, None)]	0
embedding_1 (Embedding)	(None, None, 256)	5120000
bidirectional_1 (Bidirectional)	(None, 64)	73984
dropout_1 (Dropout)	(None, 64)	0
dense_1 (Dense)	(None, 1)	65

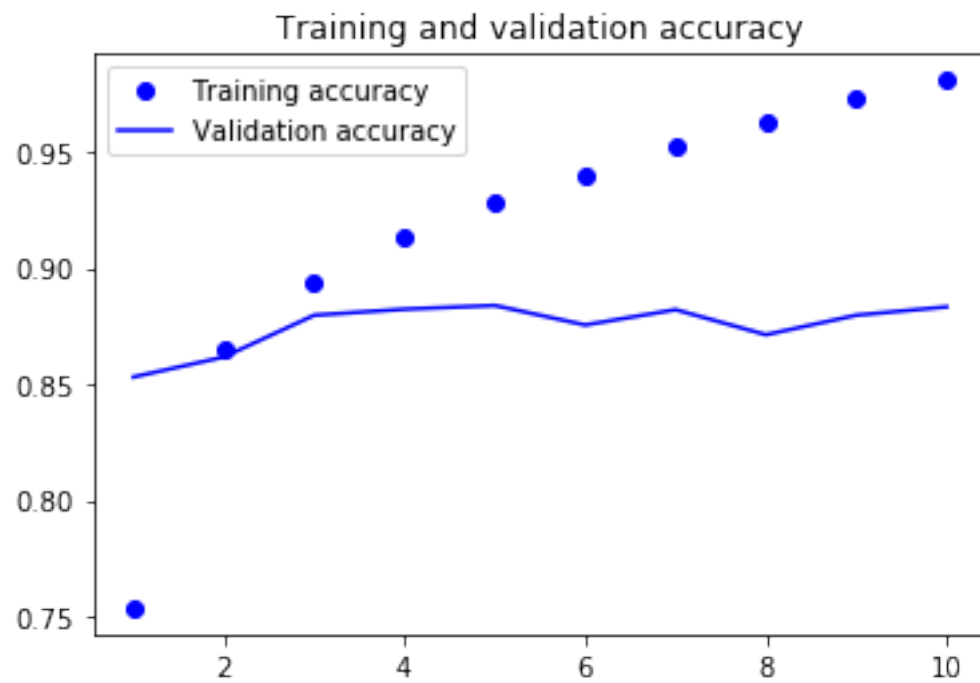
Total params: 5194049 (19.81 MB)
Trainable params: 5194049 (19.81 MB)
Non-trainable params: 0 (0.00 Byte)

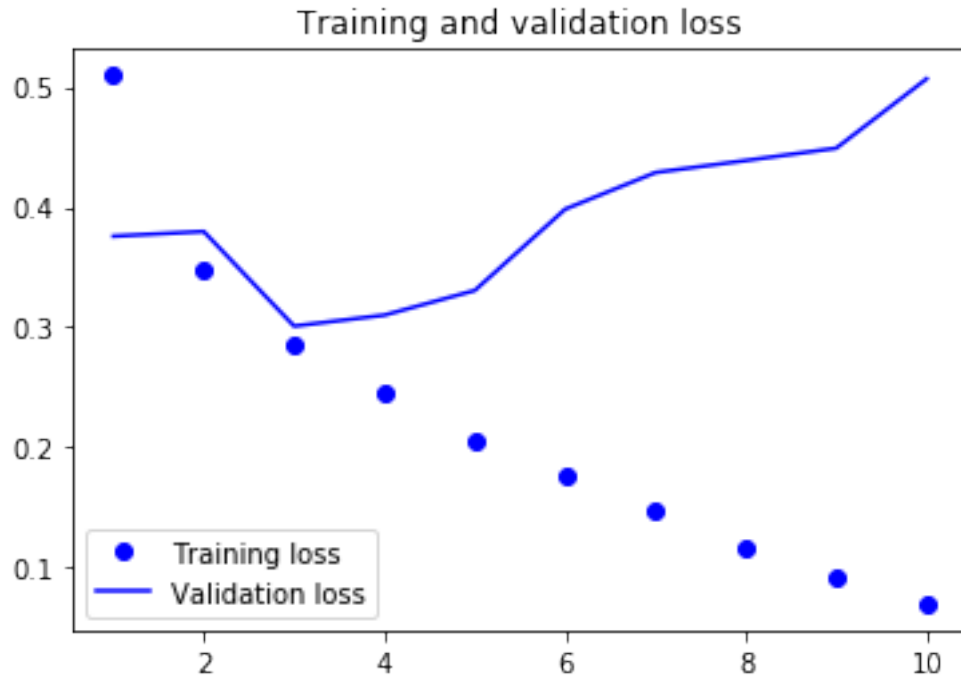
```
-----  
Epoch 1/10  
1250/1250 [=====] - 103s 80ms/step - loss: 0.5097 -  
accuracy: 0.7536 - val_loss: 0.3757 - val_accuracy: 0.8530  
Epoch 2/10  
1250/1250 [=====] - 47s 37ms/step - loss: 0.3467 -  
accuracy: 0.8655 - val_loss: 0.3797 - val_accuracy: 0.8616  
Epoch 3/10  
1250/1250 [=====] - 38s 30ms/step - loss: 0.2845 -  
accuracy: 0.8938 - val_loss: 0.3009 - val_accuracy: 0.8796  
Epoch 4/10  
1250/1250 [=====] - 36s 29ms/step - loss: 0.2453 -  
accuracy: 0.9133 - val_loss: 0.3100 - val_accuracy: 0.8822  
Epoch 5/10  
1250/1250 [=====] - 35s 28ms/step - loss: 0.2054 -  
accuracy: 0.9286 - val_loss: 0.3306 - val_accuracy: 0.8838  
Epoch 6/10  
1250/1250 [=====] - 34s 27ms/step - loss: 0.1763 -  
accuracy: 0.9394 - val_loss: 0.3982 - val_accuracy: 0.8754  
Epoch 7/10  
1250/1250 [=====] - 34s 27ms/step - loss: 0.1480 -  
accuracy: 0.9528 - val_loss: 0.4287 - val_accuracy: 0.8820  
Epoch 8/10  
1250/1250 [=====] - 33s 26ms/step - loss: 0.1173 -  
accuracy: 0.9631 - val_loss: 0.4387 - val_accuracy: 0.8712  
Epoch 9/10  
1250/1250 [=====] - 33s 27ms/step - loss: 0.0923 -  
accuracy: 0.9736 - val_loss: 0.4490 - val_accuracy: 0.8796  
Epoch 10/10  
1250/1250 [=====] - 33s 27ms/step - loss: 0.0697 -  
accuracy: 0.9808 - val_loss: 0.5067 - val_accuracy: 0.8832  
1563/1563 [=====] - 17s 10ms/step - loss: 0.3221 -  
accuracy: 0.8642  
Test acc: 0.864
```

Plotting the results

```
[8]: acc = history.history["accuracy"]  
val_acc = history.history["val_accuracy"]  
loss = history.history["loss"]  
val_loss = history.history["val_loss"]  
epochs = range(1, len(acc) + 1)  
plt.plot(epochs, acc, "bo", label="Training accuracy")  
plt.plot(epochs, val_acc, "b", label="Validation accuracy")  
plt.title("Training and validation accuracy")
```

```
plt.legend()
plt.figure()
plt.plot(epochs, loss, "bo", label="Training loss")
plt.plot(epochs, val_loss, "b", label="Validation loss")
plt.title("Training and validation loss")
plt.legend()
plt.show()
```





1.3 Using an Embedding layer with masking

- In TextVectorization: max_length = output_sequence_length
- Longer sequences truncated, shorter sequences padded with zeros -> fade out the internal state of RNNs
- Masking tells RNNs to skip meaningless iterations

```
[9]: inputs = keras.Input(shape=(None,), dtype="int64")
embedded = layers.Embedding(
    input_dim=max_tokens, output_dim=256, mask_zero=True)(inputs)
x = layers.Bidirectional(layers.LSTM(32))(embedded)
x = layers.Dropout(0.5)(x)
outputs = layers.Dense(1, activation="sigmoid")(x)
model = keras.Model(inputs, outputs)
model.compile(optimizer="rmsprop",
              loss="binary_crossentropy",
              metrics=["accuracy"])
model.summary()

callbacks = [
    keras.callbacks.ModelCheckpoint("embeddings_bidir_gru_with_masking.h5",
                                    save_best_only=True)
]
history = model.fit(int_train_ds, validation_data=int_val_ds, epochs=5,
                    ↪callbacks=callbacks)
```

```
model = keras.models.load_model("embeddings_bidir_gru_with_masking.h5")
print(f"Test acc: {model.evaluate(int_test_ds)[1]:.3f}")
```

Model: "model_2"

Layer (type)	Output Shape	Param #
input_3 (InputLayer)	[(None, None)]	0
embedding_2 (Embedding)	(None, None, 256)	5120000
bidirectional_2 (Bidirectional)	(None, 64)	73984
dropout_2 (Dropout)	(None, 64)	0
dense_2 (Dense)	(None, 1)	65

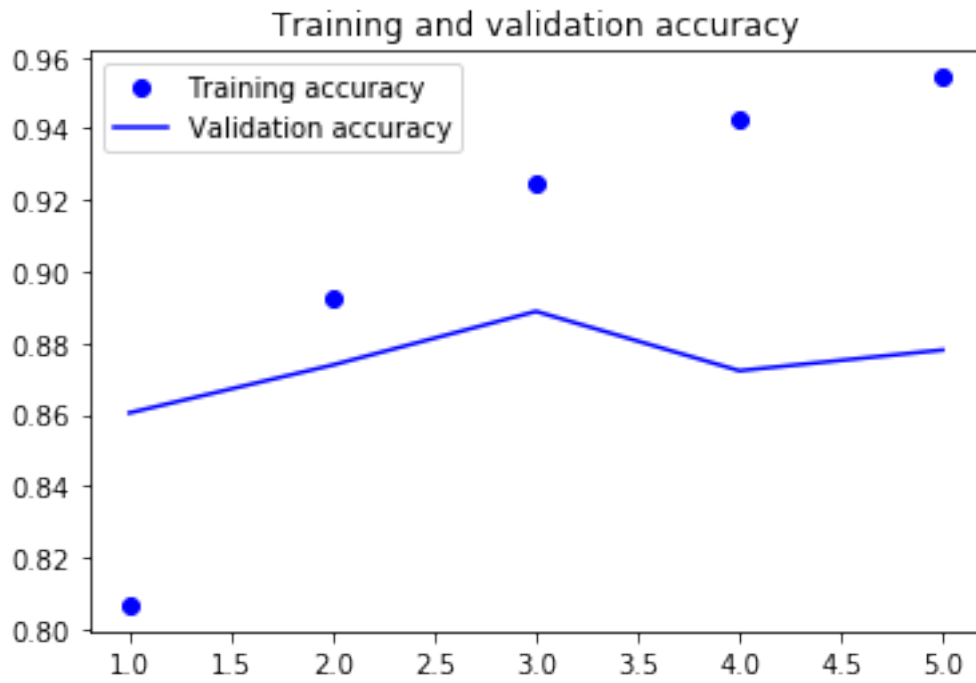
```
=====  
Total params: 5194049 (19.81 MB)  
Trainable params: 5194049 (19.81 MB)  
Non-trainable params: 0 (0.00 Byte)
```

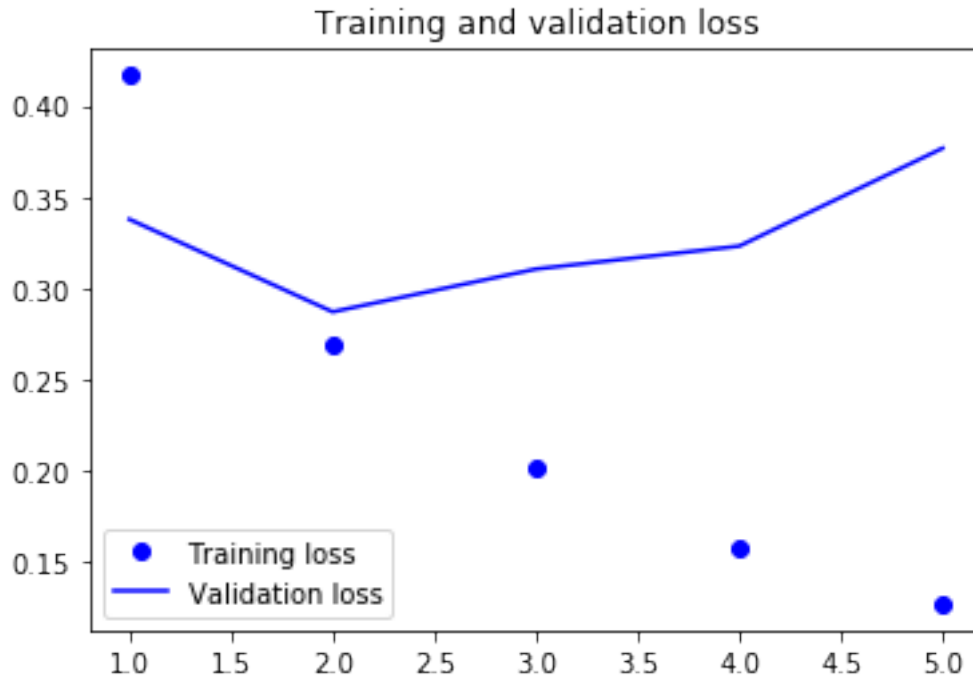
```
-----  
Epoch 1/5  
1250/1250 [=====] - 88s 66ms/step - loss: 0.4167 -  
accuracy: 0.8066 - val_loss: 0.3377 - val_accuracy: 0.8604  
Epoch 2/5  
1250/1250 [=====] - 41s 33ms/step - loss: 0.2683 -  
accuracy: 0.8924 - val_loss: 0.2869 - val_accuracy: 0.8738  
Epoch 3/5  
1250/1250 [=====] - 37s 30ms/step - loss: 0.2012 -  
accuracy: 0.9243 - val_loss: 0.3104 - val_accuracy: 0.8888  
Epoch 4/5  
1250/1250 [=====] - 35s 28ms/step - loss: 0.1574 -  
accuracy: 0.9428 - val_loss: 0.3230 - val_accuracy: 0.8722  
Epoch 5/5  
1250/1250 [=====] - 34s 28ms/step - loss: 0.1266 -  
accuracy: 0.9543 - val_loss: 0.3767 - val_accuracy: 0.8780  
1563/1563 [=====] - 18s 10ms/step - loss: 0.3019 -  
accuracy: 0.8686  
Test acc: 0.869
```

Plotting the results

```
[10]: acc = history.history["accuracy"]  
val_acc = history.history["val_accuracy"]  
loss = history.history["loss"]  
val_loss = history.history["val_loss"]
```

```
epochs = range(1, len(acc) + 1)
plt.plot(epochs, acc, "bo", label="Training accuracy")
plt.plot(epochs, val_acc, "b", label="Validation accuracy")
plt.title("Training and validation accuracy")
plt.legend()
plt.figure()
plt.plot(epochs, loss, "bo", label="Training loss")
plt.plot(epochs, val_loss, "b", label="Validation loss")
plt.title("Training and validation loss")
plt.legend()
plt.show()
```





1.4 Using pretrained word embeddings

- Little training data
- Like using a pretrained ConvNet for CV
- Load embedding vectors from a precomputed embedding space
- Hard-code linguistic/statistical model of word meaning
- Generic embedding layer does not encode linguistic structure

Parsing the GloVe word-embeddings file

```
[11]: import numpy as np
path_to_glove_file = "glove.6B.100d.txt"

embeddings_index = {}
with open(path_to_glove_file) as f:
    for line in f:
        word, coefs = line.split(maxsplit=1)
        coefs = np.fromstring(coefs, "f", sep=" ")
        embeddings_index[word] = coefs

print(f"Found {len(embeddings_index)} word vectors.")
```

Found 400000 word vectors.

Preparing the GloVe word-embeddings matrix

```
[12]: embedding_dim = 100

vocabulary = text_vectorization.get_vocabulary()
word_index = dict(zip(vocabulary, range(len(vocabulary))))

embedding_matrix = np.zeros((max_tokens, embedding_dim))
for word, i in word_index.items():
    if i < max_tokens:
        embedding_vector = embeddings_index.get(word)
        if embedding_vector is not None:
            embedding_matrix[i] = embedding_vector

embedding_layer = layers.Embedding(
    max_tokens,
    embedding_dim,
    embeddings_initializer=keras.initializers.Constant(embedding_matrix),
    trainable=False,
    mask_zero=True,
)
```

Using a pretrained word Embedding layer

- Not helpful because the training dataset is sufficiently large

```
[13]: inputs = keras.Input(shape=(None,), dtype="int64")
embedded = embedding_layer(inputs)
x = layers.Bidirectional(layers.LSTM(32))(embedded)
x = layers.Dropout(0.5)(x)
outputs = layers.Dense(1, activation="sigmoid")(x)
model = keras.Model(inputs, outputs)
model.compile(optimizer="rmsprop",
              loss="binary_crossentropy",
              metrics=["accuracy"])
model.summary()

callbacks = [
    keras.callbacks.ModelCheckpoint("glove_embeddings_sequence_model_weights.
    ↪h5",
                                save_best_only=True,
                                save_weights_only=True,
                                monitor="val_loss"
                                )
]
history = model.fit(int_train_ds, validation_data=int_val_ds, epochs=10,
    ↪callbacks=callbacks)
```

Model: "model_3"

Layer (type)	Output Shape	Param #
--------------	--------------	---------

```

=====
input_4 (InputLayer)      [(None, None)]          0

embedding_3 (Embedding)   (None, None, 100)       2000000

bidirectional_3 (Bidirecti (None, 64)              34048
onal)

dropout_3 (Dropout)       (None, 64)              0

dense_3 (Dense)           (None, 1)               65

=====
Total params: 2034113 (7.76 MB)
Trainable params: 34113 (133.25 KB)
Non-trainable params: 2000000 (7.63 MB)

-----
Epoch 1/10
1250/1250 [=====] - 39s 27ms/step - loss: 0.5508 -
accuracy: 0.7149 - val_loss: 0.4830 - val_accuracy: 0.7574
Epoch 2/10
1250/1250 [=====] - 32s 26ms/step - loss: 0.4194 -
accuracy: 0.8110 - val_loss: 0.4803 - val_accuracy: 0.7652
Epoch 3/10
1250/1250 [=====] - 32s 26ms/step - loss: 0.3734 -
accuracy: 0.8403 - val_loss: 0.3395 - val_accuracy: 0.8556
Epoch 4/10
1250/1250 [=====] - 32s 26ms/step - loss: 0.3371 -
accuracy: 0.8598 - val_loss: 0.3387 - val_accuracy: 0.8542
Epoch 5/10
1250/1250 [=====] - 32s 25ms/step - loss: 0.3123 -
accuracy: 0.8717 - val_loss: 0.3146 - val_accuracy: 0.8720
Epoch 6/10
1250/1250 [=====] - 32s 25ms/step - loss: 0.2927 -
accuracy: 0.8806 - val_loss: 0.2938 - val_accuracy: 0.8790
Epoch 7/10
1250/1250 [=====] - 32s 25ms/step - loss: 0.2775 -
accuracy: 0.8871 - val_loss: 0.2973 - val_accuracy: 0.8804
Epoch 8/10
1250/1250 [=====] - 31s 25ms/step - loss: 0.2586 -
accuracy: 0.8959 - val_loss: 0.3038 - val_accuracy: 0.8872
Epoch 9/10
1250/1250 [=====] - 31s 25ms/step - loss: 0.2417 -
accuracy: 0.9046 - val_loss: 0.2902 - val_accuracy: 0.8914
Epoch 10/10
1250/1250 [=====] - 32s 25ms/step - loss: 0.2277 -
accuracy: 0.9093 - val_loss: 0.2977 - val_accuracy: 0.8846

```

Plotting the results

```
[14]: acc = history.history["accuracy"]
      val_acc = history.history["val_accuracy"]
      loss = history.history["loss"]
      val_loss = history.history["val_loss"]
      epochs = range(1, len(acc) + 1)
      plt.plot(epochs, acc, "bo", label="Training accuracy")
      plt.plot(epochs, val_acc, "b", label="Validation accuracy")
      plt.title("Training and validation accuracy")
      plt.legend()
      plt.figure()
      plt.plot(epochs, loss, "bo", label="Training loss")
      plt.plot(epochs, val_loss, "b", label="Validation loss")
      plt.title("Training and validation loss")
      plt.legend()
      plt.show()
```

